

Capítulo 9: Subprogramas.

Atividades Práticas

Atividade Prática — LNPG

Capítulo 9 — Subprogramas

Base conceitual: Capítulo 9 do livro *Conceitos de Linguagens de Programação* de Robert W. Sebesta.

Tema central da atividade:

- modularização;
 - subprogramas;
 - passagem de parâmetros;
 - retorno de valores;
 - coesão e reutilização;
 - diferenças entre passagem por valor e referência em Java.
-

Objetivos da Atividade

Ao final da atividade, o aluno deverá ser capaz de:

- identificar partes de um programa que podem ser transformadas em subprogramas;
 - criar métodos/funções com parâmetros e retorno adequados;
 - compreender reutilização e modularização;
 - aplicar passagem de parâmetros em Java;
 - analisar efeitos colaterais em chamadas de métodos;
 - comparar organização e legibilidade de programas monolíticos vs modularizados.
-

Organização no GitHub

Cada aluno deverá criar um repositório individual seguindo o padrão:

None

`lnpg-cap9-subprogramas-NOME-SOBRENOME`

Exemplo:

None

`lnpg-cap9-subprogramas-leo-silva`

Estrutura obrigatória:

None

`README.md`
`tarefa1-java/`
`tarefa2-python/`
`tarefa3-java-parametros/`
`tarefa4-java-referencia/`
`tarefa5-livre/`

Entrega

O `README.md` principal deve conter:

- nome completo;
- turma;
- descrição breve de cada tarefa;
- instruções de execução;
- comparação entre:
 - versão monolítica;
 - versão modularizada.

Os alunos devem comentar:

- legibilidade;

- reutilização;
 - facilidade de manutenção;
 - clareza do fluxo;
 - tamanho dos métodos;
 - coesão.
-

Tarefa 1 — Modularização em Java

Objetivo

Dividir um programa grande em subprogramas.

Problema

Implemente um sistema de controle acadêmico em Java que:

- leia o nome de 5 alunos;
- leia 3 notas para cada aluno;
- calcule média;
- determine situação:
 - aprovado;
 - recuperação;
 - reprovado;
- mostre relatório final.

Requisitos

O programa inicialmente deve ser criado em uma única classe com um único método `main`.

Depois, o aluno deve refatorar o sistema criando subprogramas adequados.

Subprogramas esperados

Exemplos:

Java

```
lerAluno()  
lerNotas()  
calcularMedia()  
determinarSituacao()  
imprimirRelatorio()
```

Requisitos técnicos

- usar parâmetros corretamente;
 - usar retorno quando apropriado;
 - evitar repetição de código;
 - justificar no README as decisões de modularização.
-

Tarefa 2 — Modularização em Python

Objetivo

Identificar responsabilidades independentes e transformá-las em funções.

Problema

Implemente um sistema de vendas em Python que:

- leia produtos;
- leia quantidade;
- leia preço unitário;
- calcule subtotal;
- calcule desconto:
 - 5% acima de R\$ 200;
 - 10% acima de R\$ 500;
- calcule total final;
- imprima cupom formatado.

Requisitos

Criar inicialmente uma versão “monolítica”.

Depois refatorar utilizando funções.

Funções esperadas

Exemplos:

Python

```
ler_produto()  
calcular_subtotal()  
calcular_desconto()  
calcular_total()  
imprimir_cupom()
```

Discussão obrigatória

No README:

- quais partes eram repetitivas;
 - quais partes ficaram mais reutilizáveis;
 - impacto da modularização na legibilidade.
-

Tarefa 3 — Passagem de Parâmetros por Valor em Java

Objetivo

Compreender passagem por valor de tipos primitivos.

Problema

Crie um programa Java contendo:

Java

```
public static void alterarNumero(int x)
```

Dentro do método:

- altere o valor de `x`;
- imprima o valor antes e depois da alteração.

No `main`:

- declare uma variável inteira;
- passe para o método;
- imprima o valor antes e depois da chamada.

O aluno deve explicar

- por que o valor original não mudou;
- o que significa “passagem por valor”;
- qual valor realmente foi copiado.

Saída esperada

O aluno deve demonstrar claramente:

- mudança local;
 - ausência de mudança externa.
-

Tarefa 4 — Objetos e Referência em Java

Objetivo

Entender o comportamento de objetos em chamadas de métodos.

Problema

Crie uma classe:

Java

```
class Produto {  
    String nome;  
    double preco;  
}
```

Depois implemente:

Java

```
public static void aplicarDesconto(Produto p)
```

O método deve:

- alterar o preço do objeto;
- imprimir valores antes e depois.

No `main`:

- criar objeto;
- chamar método;
- imprimir novamente o objeto.

Parte obrigatória

O aluno deve responder:

1. Java possui passagem por referência verdadeira?
2. O que exatamente é copiado na chamada?
3. Por que alterações no objeto permanecem após a chamada?

Discussão esperada

Os alunos devem perceber:

- Java sempre usa passagem por valor;
- no caso de objetos, o valor copiado é a referência.

Tarefa 5 — Projeto Livre com

Subprogramas

Objetivo

Projetar um pequeno sistema modularizado.

O aluno deve escolher um dos temas

- sistema bancário simples;
- agenda de contatos;
- jogo simples;
- calculadora científica;
- sistema de estoque;
- conversor de unidades;
- gerenciamento de tarefas.

Requisitos mínimos

O programa deve possuir:

- pelo menos 6 subprogramas;
- pelo menos 2 funções com retorno;
- passagem de parâmetros;
- reutilização de funções;
- separação clara de responsabilidades.

Obrigatório

Adicionar no README:

- diagrama simples das chamadas;
 - justificativa da divisão dos subprogramas;
 - dificuldades encontradas;
 - vantagens percebidas da modularização.
-

